

Tworzenie aplikacji webowych

Wprowadzenie

STUDIA PODYPLOMOWE

Data Science w zastosowaniach biznesowych

Uniwersytet Warszawski, Wydział Nauk Ekonomicznych



UNIwersYTET
WARSAWski



WYDZIAŁ NAUK
EKONOMICZNYCH

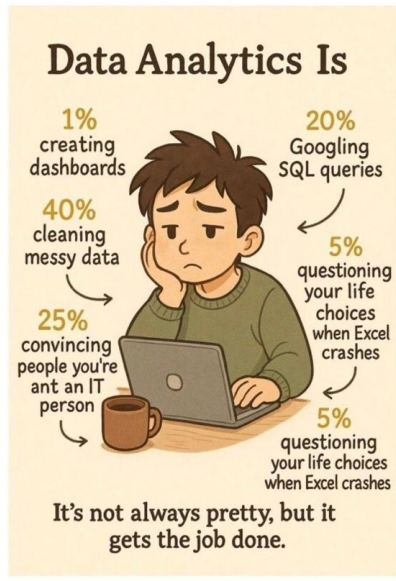
Ewa Weychert, PhD

Copyright © 2025 WNE UW. All rights reserved.

Plan zajęć

Nr	Godzina	Niedziela
1	9:00 – 9:50	1. Wstęp do Webb App
	9:50 – 10:00	<i>przerwa</i>
2	10:00 – 10:50	2. Wstęp do Streamlit
	10:50 – 11:00	<i>przerwa</i>
3	11:00 – 11:50	Różne Templatey w Streamlit
	11:50 – 12:00	<i>przerwa</i>
4	12:00 – 12:50	4. SQL and Streamlit
	12:50 – 14:00	<i>obiad</i>
5	14:00 – 14:50	5. Zaawansowany Streamlit
	14:50 – 15:00	<i>przerwa</i>
6	15:00 – 15:50	6. Shiny for Python
	15:50 – 16:00	
7	16:00 – 17:00	7. Shiny for Python

Tworzenie aplikacji webowych



Gdzie znajduje się Web app w waszych umiejętnościach?

Z tym się nie zgadzam ;)

Wstęp – Tworzenie aplikacji webowych

1 Wstęp

Czym jest aplikacja webowa?

Definicja

- Program działający w przeglądarce internetowej
- Nie wymaga instalacji po stronie użytkownika
- Dostępna z każdego urządzenia z internetem
- Aktualizowana centralnie na serwerze

Komunikacja

- Klient wysyła **request** (żądanie)
- Serwer zwraca **response** (odpowiedź)
- Protokół HTTP/HTTPS

Architektura

- **Frontend** – co widzi użytkownik
 - HTML – struktura
 - CSS – wygląd
 - JavaScript – interakcja
- **Backend** – logika aplikacji
 - Python, Django, Flask...
 - Przetwarzanie danych
 - Autoryzacja użytkowników
- **Baza danych**
 - PostgreSQL, SQLite, MySQL
 - Przechowywanie danych

Aplikacja webowa – przykłady

Znane aplikacje webowe

- Gmail, Google Docs
- Facebook, Twitter
- Allegro, Amazon
- Jupyter Notebook
- Tableau, Power BI

Dla Data Scientists

- Interaktywne dashboardy
- Interfejsy do modeli ML
- Narzędzia do eksploracji danych

Statyczna strona vs aplikacja

Strona statyczna:

Ten sam content dla każdego użytkownika

Aplikacja webowa:

Content zmienia się dynamicznie w zależności od użytkownika, danych i interakcji

Przykład Data Science

Upload pliku CSV → aplikacja przetwarza dane → wyświetla interaktywny wykres

HTTP – jak działa request i response?

Krok po kroku:

- 1 Wpisujesz adres w przeglądarce
- 2 Przeglądarka wysyła **request** do serwera
- 3 Serwer odbiera request i go przetwarza
- 4 Serwer odsyła **response** z danymi
- 5 Przeglądarka renderuje stronę

Metody HTTP:

- **GET** – pobierz dane (np. otwórz stronę)
- **POST** – wyślij dane (np. formularz)
- **PUT** – zaktualizuj dane
- **DELETE** – usuń dane

Przykład requestu

```
GET /dane?rok=2024 HTTP/1.1
Host: api.strona.pl
Accept: application/json
```

Przykład response

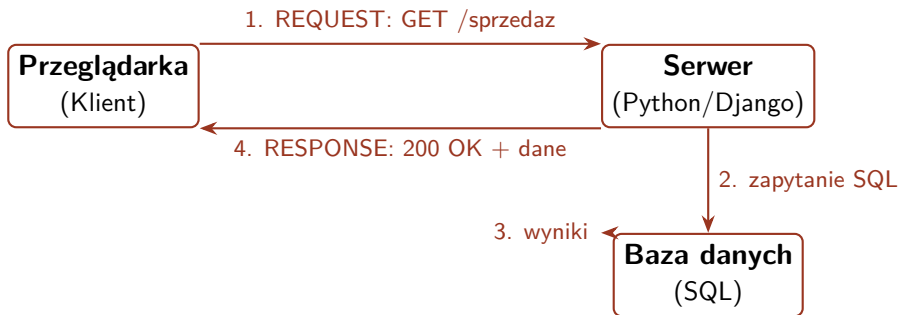
```
HTTP/1.1 200 OK
Content-Type: application/json

{"rok": 2024,
 "sprzedaz": 15000}
```

Kody odpowiedzi

```
200 – OK, sukces
404 – nie znaleziono
500 – błąd serwera
401 – brak autoryzacji
```

HTTP – schemat komunikacji



W Streamlit i Shiny ten schemat działa **automatycznie** – framework obsługuje requesty za Ciebie

Co to jest API?

API – Application Programming Interface

- Sposób komunikacji między programami
- Jeden program **pyta**, drugi **odpowiada**
- Nie musisz wiedzieć jak działa środek
- Tylko znać zasady komunikacji

Analogia – restauracja:

- Ty = aplikacja kliencka
- Kelner = API
- Kuchnia = serwer z danymi
- Menu = dokumentacja API
- Zamawiasz z menu → dostajesz danie

Przykłady popularnych API

- **OpenWeatherMap** – dane pogodowe
- **Twitter/X API** – tweety
- **Google Maps API** – mapy
- **Eurostat API** – dane statystyczne
- **NBP API** – kursy walut
- **OpenAI API** – modele AI

W Data Science

Pobierasz dane z API → przetwarzasz w pandas
→ wyświetlasz w Streamlit

REST API – jak działa w praktyce?

REST – zasady:

- Komunikacja przez HTTP
- Każdy zasób ma swój **URL** (endpoint)
- Dane przesyłane w formacie **JSON**
- Bezstanowe – każdy request niezależny

Przykład – NBP API:

```
GET  
api.nbp.pl/api/exchangerates  
/rates/a/eur/
```

Odpowiedź: kurs EUR w JSON

Python – wywołanie REST API

```
import requests  
  
url = "https://api.nbp.pl  
/api/exchangerates  
/rates/a/eur/"  
  
response = requests.get(url)  
  
data = response.json()  
  
kurs = data["rates"]  
        [0]["mid"]  
  
print(f"EUR: {kurs} PLN")
```

Zapamiętaj

requests to standardowa biblioteka Pythona do komunikacji z API

JSON – format danych w internecie

JSON – JavaScript Object Notation

- Uniwersalny format wymiany danych
- Czytelny dla człowieka i maszyny
- Używany przez **99% API** w internecie
- Bardzo podobny do słownika w Pythonie

Typy danych w JSON:

- string – "tekst"
- number – 42 lub 3.14
- boolean – true / false
- array – [1, 2, 3]
- object – {"klucz": "wartość"}
- null – brak wartości

Przykład JSON

```
{
  "imie": "Anna",
  "wiek": 32,
  "aktywna": true,
  "wydzial": "Data Science",
  "oceny": [4, 5, 4, 5],
  "adres": {
    "miasto": "Warszawa",
    "kod": "00-001"
  }
}
```

JSON w Pythonie

```
import json
data = json.loads(tekst)
print(data["imie"])
# Anna
```

JSON vs słownik Python – bliźniacy!

Cecha	JSON	Python dict
Wygląd	<code>{"a": 1}</code>	<code>{"a": 1}</code>
Klucze	zawsze string	dowolny typ
Wartość logiczna	true/false	True/False
Brak wartości	null	None
Konwersja	<code>json.loads()</code>	<code>json.dumps()</code>
Użycie	przesyłanie danych	praca w kodzie

Konwersja w obie strony

```
import json

# JSON (tekst) → Python dict
data = json.loads('{"imie": "Anna"}')

# Python dict → JSON (tekst)
tekst = json.dumps({"imie": "Anna"})
```

Dlaczego Data Scientist potrzebuje aplikacji webowych?

Problem bez aplikacji webowej

- Model ML działa tylko lokalnie na laptopie
- Wyniki w statycznym pliku Excel/PDF
- Klient nie może sam eksplorować danych
- Każda aktualizacja = nowy plik do wysłania
- Brak interakcji z modelem w czasie rzeczywistym

Rozwiązanie – aplikacja webowa

- Model dostępny dla każdego przez przeglądarkę
- Interaktywne wykresy i filtry
- Dane aktualizują się automatycznie
- Jeden link zamiast dziesiątek plików
- Użytkownik sam zadaje pytania danym

Zastosowania w praktyce biznesowej

Dashboard sprzedaży

Menedżer loguje się i widzi aktualne KPI, trendy sprzedaży, prognozy modelu ML – bez pomocy analityka

Eksploracja danych

Upload pliku CSV → automatyczna analiza → interaktywne wykresy bez znajomości Pythona

Narzędzie do predykcji

Pracownik wpisuje dane klienta → aplikacja zwraca wynik modelu (np. ryzyko odejścia, scoring kredytowy)

Automatyczny raport

Aplikacja generuje raport PDF/HTML co tydzień i wysyła do interesariuszy – zero ręcznej pracy

Data Scientist bez vs z aplikacją webową

Zadanie	Bez aplikacji	Z aplikacją
Udostępnienie modelu	e-mail z kodem	link do strony
Aktualizacja wyników	nowy plik Excel	automatyczna
Odbiorca analizy	tylko techniczny	każdy w firmie
Interakcja z danymi	brak	filtry, suwaki
Czas wdrożenia	tygodnie	godziny
Wymagania odbiorcy	Python, R	tylko przeglądarka

Aplikacja webowa to **most między Data Scientistem a biznesem**

Przegląd technologii – Frameworki webowe

Django

- Pełny framework (batteries included)
- Wbudowany ORM, auth, admin
- Idealne do dużych projektów
- Krzywa uczenia: wysoka
- Instagram, Pinterest, Mozilla

Flask

- Mikro-framework, bardzo lekki
- Pełna elastyczność
- Dobry do REST API
- Krzywa uczenia: niska
- Netflix, LinkedIn, Airbnb

FastAPI

- Nowoczesne REST API
- Automatyczna dokumentacja
- Bardzo wysoka wydajność
- Krzywa uczenia: średnia
- Popularny w ML serving

Na tym kursie

Frameworki webowe omawiamy poglądowo – **nie będziemy ich programować**

Przegląd technologii – Narzędzia dla Data Scientists

Streamlit

- Czysty Python, zero HTML
- Bardzo szybki start
- Automatyczny re-render
- Idealne do prototypów
- Duża społeczność

Shiny for Python

- Port R Shiny na Pythona
- Model reaktywny
- Tryb Express i Core
- Profesjonalne UI
- Świetny do raportów

Dash (Plotly)

- Zaawansowane wykresy
- Komponenty React pod spodem
- Dobry do złożonych dashboardów
- Krzywa uczenia: średnia
- Popularny w finansach

Na tym kursie

Skupiamy się na **Streamlit** i **Shiny for Python**

Kiedy użyć której technologii?

Potrzeba	Django	Flask	FastAPI	Streamlit	Shiny
Szybki prototyp	–	+	+	+++	++
Produkcja	+++	++	++	+	+
Baza danych	+++	+	+	–	–
Dashboardy	+	+	–	+++	+++
REST API	+	++	+++	–	–
Bez HTML/CSS	–	–	–	+++	+++
Dla Data Scientists	+	+	+	+++	+++

+++ idealne ++ dobre + możliwe – niezalecane

Ekosystem Pythona dla aplikacji webowych

Warstwa danych

- **pandas** – przetwarzanie danych
- **SQLAlchemy** – bazy danych
- **scikit-learn** – modele ML

Wizualizacja

- **plotly** – interaktywne wykresy
- **matplotlib** – statyczne wykresy
- **altair** – deklaratywne wykresy

Deployment

- **Streamlit Cloud** – darmowy hosting
- **Shinyapps.io** – hosting Shiny
- **Heroku / Railway** – platformy PaaS
- **Docker** – konteneryzacja

Narzędzia pomocnicze

- **Git** – kontrola wersji
- **VS Code** – edytor kodu
- **pip / venv** – zarządzanie pakietami

Streamlit

- Tworzenie aplikacji w czystym Pythonie
- Brak znajomości HTML/CSS/JS
- Idealny do szybkiego prototypowania
- Natywna integracja z pandas, matplotlib, plotly
- Deployment jedną komendą

Przykład

```
import streamlit as st
st.title("Hello!")
st.write(df)
```

Shiny for Python

- Port popularnego pakietu R Shiny
- Model reaktywny – UI aktualizuje się automatycznie
- Dwa tryby: **Express** (prosty) i **Core** (zaawansowany)
- Świetny do interaktywnych raportów
- Integracja z plotly, matplotlib, pandas

Przykład

```
from shiny.express
import input, render

@render.plot
def plot():
    df.plot()
```

Porównanie technologii

Cecha	Django	Streamlit	Shiny
Krzywa uczenia	wysoka	niska	średnia
Baza danych	tak	nie	nie
Reaktywność UI	ręczna	automatyczna	automatyczna
Dla Data Scientists	średnio	tak	tak
Produkcja	tak	ograniczona	ograniczona
HTML/CSS wymagane	tak	nie	nie

Co będzie potrzebne?

Oprogramowanie

- Python 3.10+
- VS Code lub PyCharm
- Przeglądarka internetowa

Biblioteki Python

- streamlit
- shiny
- pandas
- plotly

Instalacja

```
pip install streamlit shiny pandas plotly
```

Co będzie potrzebne?

Oprogramowanie

- Python 3.10+
- VS Code lub PyCharm
- Przeglądarka internetowa

Biblioteki Python

- streamlit
- shiny
- pandas
- plotly

Instalacja

```
pip install streamlit shiny pandas plotly
```


HTML – czym jest?

- **HyperText Markup Language**
- Język opisu **struktury** strony
- Nie jest językiem programowania – to znaczniki
- Przeglądarka czyta HTML i renderuje stronę
- Każda strona internetowa zbudowana jest z HTML

Podstawowe tagi:

- `<h1>...<h6>` – nagłówki
- `<p>` – paragraf
- `<div>` – kontener blokowy
- `<span>` – kontener liniowy
- `<a>` – link
- `<img>` – obrazek

Przykład HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title>Strona</title>
  </head>
  <body>
    <h1>Witaj!</h1>
    <p>To jest paragraf.</p>
    <a href="url">Link</a>
    
  </body>
</html>
```

HTML – listy ul i ol

Lista nieuporządkowana :

- Punkty bez numeracji
- Używana do nawigacji, menu, cech produktu
- Każdy element to

```
<ul>
  <li>Streamlit</li>
  <li>Shiny</li>
  <li>Dash</li>
</ul>
```

Renderuje się jako:

- Streamlit
- Shiny
- Dash

Lista uporządkowana :

- Punkty z numeracją
- Używana do kroków, instrukcji, rankingów
- Każdy element to

```
<ol>
  <li>Zainstaluj Python</li>
  <li>Zainstaluj Streamlit</li>
  <li>Uruchom app.py</li>
</ol>
```

Renderuje się jako:

1. Zainstaluj Python
2. Zainstaluj Streamlit
3. Uruchom app.py

HTML – tagi przydatne w aplikacjach webowych

Formularze i input:

```
<form>
  <label>Imię:</label>
  <input type="text"
    name="imie">

  <label>Wiek:</label>
  <input type="number"
    name="wiek">

  <button type="submit">
    Wyślij
  </button>
</form>
```

Tabela danych:

```
<table>
  <thead>
    <tr>
      <th>Produkt</th>
      <th>Cena</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Laptop</td>
      <td>3000 zł</td>
    </tr>
  </tbody>
</table>
```

CSS – czym jest?

- **Cascading Style Sheets**

- Odpowiada za **wygląd** strony
- HTML = szkielet, CSS = ubranie
- Kolory, czcionki, rozmiary, układy
- Można zmieniać wygląd bez zmiany treści

Co można stylować:

- Kolory tła i tekstu
- Rozmiar i krój czcionki
- Marginesy i odstępy
- Układ elementów (grid, flexbox)
- Animacje i efekty hover

Przykład CSS

```
h1 {  
    color: red;  
    font-size: 24px;  
}  
  
p {  
    font-family: Arial;  
    margin: 10px;  
}  
  
.box {  
    background: blue;  
    padding: 20px;  
}
```

HTML + CSS – jak działają razem?

	HTML	CSS
Rola	Struktura i treść	Wygląd i styl
Analogia	Szkielet budynku	Elewacja i wystrój
Przykład	<code><h1>Tytuł</h1></code>	<code>color: red</code>
Bez drugiego	Działa, ale brzydko	Nie działa samodzielnie

Dlaczego to ważne dla Data Scientists?

Streamlit i Shiny generują HTML i CSS **automatycznie** – nie musisz ich znać, ale rozumiejąc podstawy możesz:

- Dostosować wygląd aplikacji
- Debugować problemy z layoutem
- Dodać własne style przez `st.markdown()` lub `ui.HTML()`

HTML, CSS i JavaScript – prosta analogia

HTML

Szkielet budynku

Mury, okna, drzwi, schody –
sama struktura bez wyglądu i
funkcji

CSS

Wystrój wnętrz

Kolor ścian, meble, tapety –
budynek wygląda ładnie ale
nadal nic nie robi

JavaScript

Elektryka i instalacje

Prąd, winda, alarm – teraz
budynek reaguje i coś robi

Bez HTML strona nie istnieje. Bez CSS jest brzydka. Bez JS nic się nie dzieje.

HTML – dosłownie jak WORD, ale dla przeglądarki

Co robisz w Wordzie:

- Piszesz tytuł → duży, pogrubiony
- Piszesz akapit → normalny tekst
- Wstawiasz zdjęcie
- Robisz listę punktowaną
- Wstawiasz tabelę

To samo robisz w HTML:

- `<h1>Tytuł</h1>`
- `<p>Akapit</p>`
- ``
- `<ul><li>punkt</li></ul>`
- `<table>...</table>`

Zasada działania tagów

Każdy tag ma **otwarcie** i **zamknięcie**:

```
<h1>To jest tytuł</h1>
      ↑ otwórz      ↑ zamknij
```

```
<p>To jest paragraf</p>
```

```
<strong>Pogrubiony</strong>
```

```
<em>Kursywa</em>
```

Wyjątek – tagi bez zamknięcia:

```

```

```
<br> – nowa linia
```

```
<hr> – pozioma linia
```

Zapamiętaj

Tag niezamknięty = błąd na stronie!

HTML – co widzi przeglądarka vs co widzisz Ty

Kod HTML (piszesz):

```
<h1>Moja strona</h1>
<p>Witaj na mojej stronie!</p>
<ul>
  <li>Python</li>
  <li>Streamlit</li>
  <li>Pandas</li>
</ul>
<a href="google.com">
  Kliknij tutaj
</a>
```

Efekt w przeglądarce (widzisz):

Moja strona

Witaj na mojej stronie!

- Python
- Streamlit
- Pandas

[Kliknij tutaj](#)

Przeglądarka **ukrywa tagi** – użytkownik widzi tylko efekt końcowy, nigdy kod HTML

CSS – jak zmienić kolor jednego przycisku?

Bez CSS – wszystko szare i brzydkie:

```
<button>Kliknij</button>
```

Efekt: szary przycisk, domyślna czcionka, brak stylu

Z CSS – ładny przycisk:

```
<button style="
  background: red;
  color: white;
  padding: 10px 20px;
  border-radius: 5px;
  font-size: 16px;
">Kliknij</button>
```

Efekt: czerwony przycisk z białym tekstem

Najważniejsze właściwości CSS:

CSS	Co robi
color	kolor tekstu
background	kolor tła
font-size	rozmiar czcionki
font-weight	pogrubienie
padding	odstęp wewnętrzny
margin	odstęp zewnętrzny
border	ramka
border-radius	zaokrąglenie
width	szerokość
display	sposób wyświetlania

CSS – padding vs margin (często mylone!)

Padding – odstęp WEWNĄTRZ elementu

```
<p style="padding: 20px; background: lightblue;">  
  Tekst z paddingiem  
</p>
```

Efekt: niebieski obszar otacza tekst z każdej strony od środka

Margin – odstęp NA ZEWNĄTRZ elementu

```
<p style="margin: 20px; background: lightblue;">  
  Tekst z marginem  
</p>
```

Efekt: przestrzeń między tym elementem a sąsiadami

Padding = odstęp między treścią a ramką **Margin** = odstęp między ramką a resztą strony

JavaScript – co to znaczy “reaguje na kliknięcie”?

HTML sam w sobie jest martwy:

- Klikasz przycisk – nic się nie dzieje
- Wpisujesz tekst – strona nie reaguje
- Nie można zmienić zawartości bez przeładowania

JavaScript ożywia stronę:

- Klikasz przycisk → pojawia się wykres
- Wybierasz datę → dane się filtrują
- Wpisujesz tekst → lista się aktualizuje
- Strona pobiera nowe dane **bez przeładowania**

Przykład – licznik kliknięć

```
<p id="licznik">0</p>
<button onclick="
    dodaj()">Kliknij</button>
```

```
<script>
let n = 0;

function dodaj() {
    n = n + 1;
    document
        .getElementById(
            "licznik")
        .innerHTML = n;
}
</script>
```

Dlaczego Data Scientist nie musi tego umieć?

Tradycyjna aplikacja webowa:

- 1 Projektant pisze HTML
- 2 Developer pisze CSS
- 3 Developer pisze JavaScript
- 4 Backend developer pisze Python
- 5 Tester sprawdza całość

Zespół 3–5 osób, tygodnie pracy

Streamlit / Shiny:

- 1 Data Scientist pisze Python
- 2 *(koniec)*

Streamlit i Shiny **automatycznie generują** HTML, CSS i JavaScript w tle
– Ty widzisz tylko Python

Czyli:

```
st.button("Kliknij")
```

↓

Streamlit generuje za Ciebie:

```
<button class="st-btn"  
  onclick="..."  
  style="...">Kliknij</button>
```

Środowiska wirtualne

Dlaczego używać venv?

- Każdy projekt ma własne zależności
- Unikasz konfliktów między wersjami pakietów
- Łatwe odtworzenie środowiska na innym komputerze
- Standard w profesjonalnych projektach

requirements.txt

```
streamlit==1.35.0  
shiny==0.10.0  
pandas==2.2.0  
plotly==5.20.0  
sqlite3==3.45.0
```

Tworzenie i użycie venv

```
# Utwórz środowisko  
python -m venv venv  
  
# Aktywuj (Mac/Linux)  
source venv/bin/activate  
  
# Aktywuj (Windows)  
venv\Scripts\activate  
  
# Zainstaluj pakiety  
pip install -r  
    requirements.txt  
  
# Zapisz zależności  
pip freeze >  
    requirements.txt
```

Przykłady użycia dla Was

Dla biznesu

- Dashboard KPI firmy
- Analiza koszyka zakupów
- Predykcja churnu klientów
- Analiza sentymentu opinii

Dla finansów

- Analiza portfela akcji
- Kalkulator kredytowy
- Dashboard kryptowalut

Dla nauki

- Eksploracja publicznego datasetu
- Wizualizacja wyników ankiety
- Interaktywna mapa danych GUS

Dla każdego

- Aplikacja do analizy własnych danych
- Generator raportów z CSV
- Prosty klasyfikator ML z UI

Typowy projekt Data Science – od danych do aplikacji

Etap	Co robisz	Narzędzie
1	Pobierasz dane (CSV, API, SQL)	pandas, requests
2	Czyścisz i przetwarzasz dane	pandas, numpy
3	Eksplorujesz dane (EDA)	matplotlib, plotly
4	Budujesz model ML	scikit-learn
5	Tworzysz aplikację webową	Streamlit / Shiny
6	Testujesz lokalnie	przeglądarka
7	Wdrażasz na serwer	Streamlit Cloud
8	Udostępniasz link klientowi	e-mail / Teams

Etapy 1–4 już znasz z poprzednich kursów.

Ten kurs uczy Cię etapów 5–8.

Materiały i zasoby

Dokumentacja oficjalna:

- docs.streamlit.io
- shiny.posit.co/py
- plotly.com/python
- pandas.pydata.org

Platformy do nauki:

- kaggle.com – datasets + kursy
- realpython.com – tutoriale
- youtube.com/@streamlit – kanał Streamlit

Gotowe szablony:

- streamlit.io/gallery
- github.com/topics/streamlit
- shinylive.io/py/examples

Materiały kursu

Wszystkie slajdy, kody i przykłady dostępne na repozytorium GitHub kursu: [github.com/\[repo-kursu\]](https://github.com/[repo-kursu])

Dziękuję za uwagę